Now, how can you pass my_var through the def Scope Gate? Once again, you have to replace the keyword with a method call. Think of the discussion about *Dynamic Methods (51)*: instead of def, you can use Module#define_method:

blocks/flat_scope_3.rb

```
my_var = "Success"

MyClass = Class.new do
  "#{my_var} in the class definition"

➤  define_method :my_method do
➤    "#{my_var} in the method"
➤  end
end

➤ MyClass.new.my_method

require_relative "../test/assertions"
assert_equals "Success in the method", MyClass.new.my_method
```

```
❮ Success in the class definition
Success in the method
```

If you replace Scope Gates with method calls, you allow one scope to see variables from another scope. Technically, this trick should be called *nested lexical scopes*, but many Ruby coders refer to it simply as "flattening the scope," meaning that the two scopes share variables as if the scopes were squeezed together. For short, you can call this spell a *Flat Scope*.

Spell: Flat Scope

Sharing the Scope

Once you know about *Flat Scopes (83)*, you can do pretty much whatever you want with scopes. For example, assume that you want to share a variable among a few methods, and you don't want anybody else to see that variable. You can do that by defining all the methods in the same Flat Scope as the variable:

blocks/shared_scope.rb

```
def define_methods
  shared = 0

  Kernel.send :define_method, :counter do
    shared
  end

  Kernel.send :define_method, :inc do |x|
    shared += x
  end
end
```